

NumPy, short for *Numerical Python*, is a foundational Python library built specifically for heavy numerical computing and data processing.

Why Python Needs NumPy

- **The Problem:** Standard Python lists store objects dynamically, causing massive overhead when managing millions of numbers. Additionally, multiplying a normal Python list by an integer repeats its elements (`[1, 2] * 2` becomes `[1, 2, 1, 2]`) rather than applying math to each element.
- **The Solution:** NumPy is written natively in **C**, making it magnitudes faster. It introduces the **NumPy Array**, which enables vectorized operations (applying a mathematical calculation to an entire dataset simultaneously without explicit loops).

Installation & Initialization

Open your terminal (**Alt + F12**) and run:

```
Bash
pip install numpy
```

In your Python script, import the library using the standard ecosystem alias `np`:

```
Python
import numpy as np
```

```
# Verify successful setup and check version
print(np.__version__)
```

2. Multi-Dimensional Arrays (`ndarray`) The core object in NumPy is the `ndarray` (n-dimensional array). It can scale across structural depths seamlessly, but must remain **homogeneous** (every single element inside the array must share the exact same data type and consistency).

```
Python
# 0-D Array (A single scalar value point)
arr_0d = np.array(42)

# 1-D Array (A single flat row vector)
arr_1d = np.array(['a', 'b', 'c'])
```

```
# 2-D Array (A matrix layout containing rows and columns)
arr_2d = np.array([[ 'a', 'b', 'c'], [ 'd', 'e', 'f']])

# Checking the structural dimension count
print(arr_2d.ndim) # Outputs: 2(if 2D array then o/p will be 2)
```

Shape Consistency Rule

If your data has inhomogeneous shapes (e.g., one row has 3 elements but another row has only 2), NumPy will crash with a `ValueError`. Always insert clean placeholders (like a space or an underscore) to keep array boundaries structurally symmetrical.

Understanding Array Shapes (`.shape`)

The `.shape` attribute returns a tuple mapping out the layout boundaries of an array as (Depth/Layers, Rows, Columns).

```
Python
# A 3D layered array setup
arr_3d = np.array([[[ 'a', 'b'], [ 'c', 'd']],
                  [[ 'e', 'f'], [ 'g', 'h']]])

print(arr_3d.shape) # Outputs: (2, 2, 2) -> 2 layers, 2 rows, 2 columns
```

Multi-Dimensional Indexing

Avoid native Python chain indexing (`arr[0][0][0]`) because it creates unnecessary intermediate array objects. Instead, use comma-separated multi-dimensional indexing inside a single set of brackets.

```
Python
# Fast multi-dimensional lookup
element = arr_3d[0, 1, 0] # Layer 0, Row 1, Column 0
```

3. Advanced Slicing Techniques Slicing expressions allow you to carve out precise chunks from your data using the format: `[start : end : step]`. Remember, the `end` index parameter is always **exclusive**.

```
Python
# Setup a 4x4 sample matrix
matrix = np.array([
```

```
[1, 2, 3, 4],  
[5, 6, 7, 8],  
[9, 10, 11, 12],  
[13, 14, 15, 16]  
)
```

Row Slicing

Python

```
print(matrix[0]) # Pulls Row 0 entirely: [1, 2, 3, 4]  
print(matrix[-1]) # Pulls the final row via negative indexing: [13, 14, 15, 16]  
print(matrix[0:3]) # Pulls Rows 0, 1, and 2 (Index 3 is excluded)  
print(matrix[::2]) # Pulls every second row starting at index 0 (Rows 0 and 2)  
print(matrix[::-1]) # Flips the vertical sequence layout (Reverses all rows)
```

Column Slicing

To isolate columns, pass a colon (`:`) into the first parameter position to select *all rows*, then specify your column constraints in the second position.

Python

```
print(matrix[:, 0]) # Pulls Column 0 across all rows: [1, 5, 9, 13]  
print(matrix[:, 0:3]) # Pulls the first 3 columns (Columns 0, 1, 2)  
print(matrix[:, ::2]) # Pulls every second column across the dataset  
print(matrix[:, ::-1]) # Flips the horizontal axis (Reverses all columns)
```

Quadrant Slicing (Combining Rows & Columns)

Python

```
# Isolating the top-left 2x2 quadrant block [[1, 2], [5, 6]]  
top_left = matrix[0:2, 0:2]
```

```
# Isolating the bottom-right 2x2 quadrant block [[11, 12], [15, 16]]  
bottom_right = matrix[2:, 2:]
```

4. Arithmetic & Vectorized Operations

NumPy changes standard math workflows by allowing you to process large-scale datasets instantly without writing nested loops.

Scalar Arithmetic

Applying a standalone standalone numerical scalar value to an array distributes that calculation across every individual element inside it.

Python

```
arr = np.array([1, 2, 3])
```

```
print(arr + 1) # Addition: [2, 3, 4]
```

```
print(arr * 3) # Multiplication: [3, 6, 9]
```

```
print(arr ** 2) # Exponentiation: [1, 4, 9]
```

Vectorized Math Functions & Constants

NumPy contains robust built-in mathematical functions that process arrays effortlessly.

Python

```
radii = np.array([1, 2, 3])
```

```
# Vectorized formula for calculating Circle Areas: Area = pi * r^2
```

```
areas = np.pi * (radii ** 2)
```

```
# Rounding utilities
```

```
print(np.round(areas, 2))
```

```
print(np.floor(areas)) # Forces downward rounding truncation
```

```
print(np.ceil(areas)) # Forces upward rounding truncation
```

Element-Wise Arithmetic

When math is performed directly between two arrays of matching size shapes, NumPy pairs up the items by position to compute them.

Python

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
print(a + b) # Computes element-to-element: [5, 7, 9]
```

```
print(a * b) # Computes element-to-element: [4, 10, 18]
```

Comparison Filters & Conditional Assignment

Python

```
scores = np.array([91, 55, 100, 73, 82, 64])
```

```
# Returns a structural mask array populated with matching booleans
```

```
print(scores >= 60) # Outputs: [True, False, True, True, True, True]
```

```
# Conditional Assignment: Catch failing grades under 60 and reset them to 0
scores[scores < 60] = 0
print(scores) # Outputs: [91, 0, 100, 73, 82, 64]
```

5. Broadcasting Rules

Broadcasting is the mechanism that allows NumPy to perform mathematical calculations between arrays of completely different structural shapes. NumPy virtually stretches or expands the smaller array dimensions to fit across the larger framework smoothly.

The Two Strict Rules of Broadcasting

Two array shapes are compatible for broadcasting *only* if they are scanned from **right to left** (trailing dimensions upward) and satisfy one of these conditions for each axis:

1. The dimensions are exactly the same size number.
2. One of the dimensions is exactly **1**.

Python

```
# Example 1: Creating a 10x10 Multiplication Table seamlessly
arr_vertical = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9], [10]]) # Shape: (10, 1)
arr_horizontal = np.array([[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]) # Shape: (1, 10)

# Broadcasting automatically expands shapes out to a unified (10, 10) matrix grid
multiplication_table = arr_vertical * arr_horizontal
```

6. Aggregate Functions

Aggregates condense collections of numerical raw values down into targeted mathematical summary metrics.

Python

```
matrix_data = np.array([[1, 2, 3], [4, 5, 6]])
```

```

print(np.sum(matrix_data)) # Sum of all elements: 21
print(np.mean(matrix_data)) # Core average mean: 3.5
print(np.min(matrix_data)) # Lowest item value: 1
print(np.max(matrix_data)) # Highest item value: 6

# Finding position index locations instead of the raw values
print(np.argmin(matrix_data)) # Index position location of minimum element: 0
print(np.argmax(matrix_data)) # Index position location of maximum element: 5

```

Targeting Specific Axes

You can isolate aggregate tracking directions cleanly by specifying an **axis** parameter:

- **axis=0**: Tracks downward across the vertical structural layout (**Columns**).
- **axis=1**: Tracks sideways across the horizontal structural layout (**Rows**).

```

Python
# Summing columns down vertically
print(np.sum(matrix_data, axis=0)) # Outputs array: [5, 7, 9]

# Summing rows across horizontally
print(np.sum(matrix_data, axis=1)) # Outputs array: [6, 15]

```

7. Boolean Indexing vs. **np.where()**

Filtering selectively retains elements that fulfill conditions.

Boolean Indexing (Flattens Arrays)

Passing a logical argument inside array brackets extracts matching entries directly. **This process will flatten multi-dimensional arrays into a flat 1D array.**

```

Python
ages = np.array([[21, 17, 19],
                 [35, 16, 65]])

# Filter with complex bitwise operators (& for And, | for Or) wrapped in parentheses
adult_filter = ages[(ages >= 18) & (ages < 65)]
print(adult_filter) # Outputs flat 1-D structure array: [21, 19, 35]

```

np.where() (Preserves Matrix Shape)

If you must maintain the original array structure, utilize the `np.where(condition, value_if_true, value_if_false)` function.

Python

```
# Swap minors under 18 out for 0, but preserve mature entries and the 2D matrix shape
clean_matrix = np.where(ages >= 18, ages, 0)
print(clean_matrix)
# Outputs:
# [[21, 0, 19],
# [35, 0, 65]]
```

8. Random Numbers & Array Operations

Modern NumPy utilizes an optimized object-oriented random number generator interface setup via `np.random.default_rng()`.

Python

```
# Initialize the modern Random Number Generator object interface
rng = np.random.default_rng()
```

Generating Random Data

Python

```
# 1. Random Integers (Low inclusive, High exclusive)
dice_roll = rng.integers(low=1, high=7)           # Standalone scalar
matrix_roll = rng.integers(low=1, high=101, size=(3,2)) # Structured 3x2 matrix array

# 2. Random Floats (Uniform distribution layout boundaries between low and high metrics)
float_matrix = rng.uniform(low=-1.0, high=1.0, size=(3,2))
```

Setting Reproducible seeds

If you need to guarantee your script generates the exact same random outputs every time it runs (vital for debugging and testing scientific model configurations), initialize your generator object with an explicit static value seed.

Python

```
rng_locked = np.random.default_rng(seed=1)
```

Shuffling Data & Making Random Choices

Python

```
# Shuffling an existing array directly in-place
cards = np.array([1, 2, 3, 4, 5])
rng.shuffle(cards)
```

```
# Making a random choice from an array of strings/emojis
fruits = np.array(["🍎", "🍊", "🍌", "🍫", "🍍"])
single_pick = rng.choice(fruits)
matrix_picks = rng.choice(fruits, size=(3,3)) # Build a randomized matrix of choices
```